

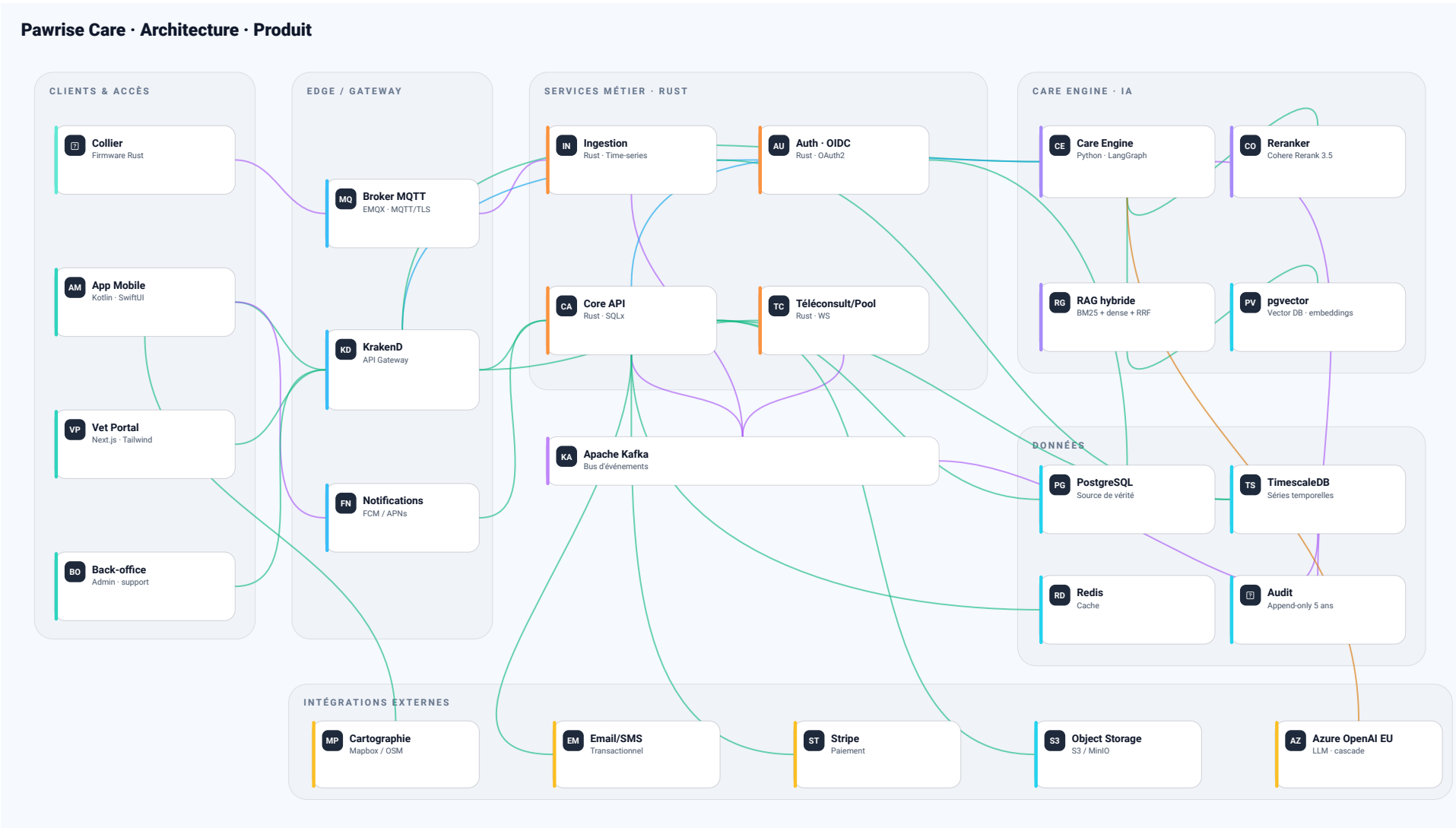
1 Architecture technique

1.1 Vue d'ensemble

L'architecture suit **quatre principes directeurs**, chacun assumé et justifié plus bas :

1. **Services découplés par domaine** : un ensemble restreint de services indépendants alignés sur les domaines métier (auth, cœur, ingestion temps-réel, téléconsultation, moteur IA), derrière une passerelle unique. Les frontières suivent les runtimes (Rust / Python) et les profils de charge, pour faire évoluer et monter en charge chaque service séparément, sans multiplier les microservices fins.
2. **Architecture événementielle** : les services communiquent via un bus (Kafka) pour se découpler et ne perdre aucun événement (ingestion, projections, alertes, audit, file de téléconsultation).
3. **IA en cage** : le Care Engine est un pipeline borné qui explique et oriente, sans jamais diagnostiquer ni écrire directement dans les données métier.
4. **Souveraineté et coût maîtrisé** : Rust économe, une seule famille de bases (PostgreSQL), hébergement UE auto-géré.

Flux : — synchrone (aller-retour) · — événement (sens unique) · — contrôle · — externe.



Plateforme & Ops (transverse) : Kubernetes auto-géré (Hetzner, Terraform) · GitOps (Argo CD) · observabilité OpenTelemetry → Prometheus / Loki / Tempo / Grafana · secrets chiffrés (SOPS). Cette couche transverse a son propre schéma de déploiement (cluster, CI/CD, observabilité, hébergement) en partie Cloud & infrastructure. Diagramme exporté du cockpit interactif : version complète et navigable sur pawrise-care.com.

Lecture par zone

- **Clients & accès** : une **app mobile native** (bien-être, localisation, alertes, chat IA, appairage BLE), un **Vet Portal web** (historique structuré + PDF normalisé) et une console **Admin**. Tout le trafic applicatif passe par **KrakenD**, une passerelle unique : un seul point d'entrée à sécuriser, qui centralise authentification, limitation de débit et routage.
- **Entrée IoT** : le **collier** parle en **MQTT sur TLS** (certificat par appareil) à un broker **EMQX**, séparé de la passerelle HTTP. MQTT est le standard IoT : pub/sub léger, QoS et mode hors-ligne gratuits, adaptés à une batterie et un réseau cellulaire instable.
- **Services métier (Rust)** : **OIDC** (identité et droits owner / vétérinaire), **Core API** (comptes, animaux, alertes), **Ingestion** (normalisation de la télémétrie), **véto & abonnement**. Chacun est un **service indépendant**, déployé et mis à l'échelle séparément selon sa charge ; ils communiquent via la passerelle et le bus d'événements plutôt que par appels directs, ce qui les garde faiblement couplés.
- **Bus d'événements (Kafka)** : la colonne vertébrale. Les services **publient** des événements ; les consommateurs (projections, Care Engine, audit, file de téléconsultation) s'y **abonnent**. Résultat : services découplés et **aucun événement perdu**.
- **IA · Care Engine (Python)** : un pipeline **LangGraph borné** qui consomme le **RAG** (pgvector) et le LLM (**Azure OpenAI UE** + reranker Cohere). Il **ne peut pas écrire** dans les données métier : il produit une orientation et, si besoin, **escalade** vers le vétérinaire via une interface contrôlée.
- **Données** : une seule famille **PostgreSQL** : transactionnel (PostgreSQL), séries temporelles de télémétrie (**TimescaleDB**) et embeddings du corpus (**pgvector**), plus **Redis** pour le cache et les sessions. Un seul moteur à opérer, sauvegarder et sécuriser.

L'IA EN CAGE · RÈGLE D'OR

Le Care Engine n'a **aucun accès en écriture** au métier et suit un pipeline déterministe : il explique, oriente et escalade, **jamais il ne diagnostique** (Code rural, art. L243-1). Cette contrainte est portée par l'architecture elle-même, pas seulement par le modèle, ce qui la rend juridiquement défendable (ADR-001).

1.2 Choix techniques justifiés

Chaque décision structurante : besoin, alternatives écartées, décision, et trade-off assumé.

Choix	Alternatives écartées	Décision & trade-off
Rust (services métier)	Node, Python, Go, C#	Perf + sûreté mémoire, empreinte minimale. Coût dev ~2,5× assumé (mitigé : périmètre Rust ciblé sur les services critiques, Python pour l'IA, workspace Cargo partagé).
Découpage en services	Monolithe unique, microservices fins	Frontières par domaine + runtime (Rust / Python) et profil de charge (ingestion, WebSocket). Montée en charge ciblée ; surcoût d'exploitation absorbé par Kubernetes, GitOps et l'observabilité.
MQTT / EMQX (entrée IoT)	HTTP, CoAP, passerelle maison	Standard IoT : pub/sub léger, QoS, offline gratuits. Un composant de plus à opérer.
Apache Kafka (bus)	RabbitMQ, appels directs	Débit, rétention, découplage. Plus gourmand en RAM (dimensionne le cluster).
PostgreSQL + TimescaleDB Mongo + Influx + Qdrant séparés + pgvector		Une seule famille : transactionnel, séries temporelles, vectoriel. Bascule Qdrant possible.

Choix	Alternatives écartées	Décision & trade-off
Pipeline LangGraph borné	Agent LLM autonome	Chaque étape testable, responsabilité juridique portée par le pipeline (ADR-001).
Azure OpenAI EU + cascade + Cohere	LLM hors UE, modèle unique	Data residency UE, ~0,03-0,05 €/conv. Dépendance Microsoft mitigée (interface LLMPProvider).
Kubernetes auto-géré + GitOps	PaaS, managé d'emblée	Économique et souverain ; plus d'ops, mitigé par l'IaC. Managé EU au scale.
OIDC maison	Auth0, Keycloak, Cognito	Souveraineté + défi formateur ; coûteux (~4-6 mois-homme), revue de sécurité obligatoire.

1.3 Scénarios & flux d'événements

Les services ne s'appellent presque jamais en direct : ils **publient** et **consomment** des événements sur Kafka, organisés en topics par domaine. Un même événement est lu par plusieurs consommateurs indépendants, qui évoluent sans se connaître : on ajoute une réaction (notification, projection, audit) sans toucher au producteur.

Topic Kafka	Producteur	Consommateurs	Charge utile
telemetry.normalized	Ingestion	TimescaleDB (série · historique)	Mesure normalisée (position, activité, température, rythme) + horodatage serveur + n° de séquence.
health.alert	Core API (règles de santé)	Notifications · Email/SMS · Care Engine	Animal, type de signal, sévérité, fenêtre d'observation.
care.escalation	Care Engine	Téléconsult/Pool (file) · Notifications	Animal, motif, résumé contextualisé, priorité.

Exemple de bout en bout : un signal faible détecté.

- Mesure** : le collier échantillonne en continu (activité, température, rythme) et publie en MQTT/TLS. En cas de coupure, il bufferise localement et rejoue à la reconnexion : aucune mesure perdue.
- Ingestion** : abonnée au broker, elle décode, **déduplique** (n° de séquence + horodatage serveur), écrit la série dans TimescaleDB et publie `telemetry.normalized` sur Kafka.
- Détection (Core API)** : le **module de règles de santé** de Core API compare les séries récentes de l'animal à ses **lignes de base** (activité, température, rythme, zones GPS). Baisse d'activité + légère hausse de température sur 48 h = **signal faible**.
- Alerte** : Core API publie `health.alert` (sévérité « faible ») ; **Notifications** et **Email/SMS** préviennent le propriétaire.
- Orientation IA** : saisi de l'alerte ou interrogé par le propriétaire, le **Care Engine** contextualise (RAG + historique) et propose une orientation prudente (surveiller, ou consulter). Il **ne pose aucun diagnostic**.
- Escalade si besoin** : si le tableau est ambigu ou s'aggrave, le Care Engine publie `care.escalation` : la demande entre dans la file de Téléconsult/Pool et le vétérinaire est notifié.
- Traçabilité** : l'intégralité de la conversation IA est enregistrée dans le journal d'audit (voir ci-dessous).

1.4 Résilience & disponibilité

- **Ingestion idempotente** : chaque trame porte un numéro de séquence ; l'Ingestion déduplique et réordonne par horodatage serveur. Un rejeu après coupure n'introduit ni doublon ni trou d'historique.
- **Collier hors-ligne** : buffer local et rejeu à la reconnexion, pensés pour un réseau cellulaire instable.
- **Dead-letter queue** : un événement non traitable part dans une file de rejeu (DLQ) au lieu de bloquer le flux ; il est supervisé puis rejoué après correctif.
- **Nouvelles tentatives bornées** : les appels sortants (LLM, paiement, cartographie) sont protégés par des retries avec backoff et des délais plafonnés.
- **Dégradation gracieuse de l'IA** : si le LLM est indisponible, le chat passe en file d'attente et toute situation à risque est **escaladée directement** vers le vétérinaire. La sécurité prime sur la fonctionnalité.
- **Sans état & mise à l'échelle** : les services Rust sont sans état (sessions dans Redis) et se répliquent horizontalement ; Kafka partitionne par animal pour paralléliser les traitements.
- **Sauvegardes & reprise** : PostgreSQL en restauration à un instant (PITR), snapshots des volumes, sauvegarde planifiée de l'état du cluster (détail en partie Cloud & infrastructure).
- **Objectifs de service** : ingestion et alertes traitées en continu ; disponibilité cible et latence p95 suivies via l'observabilité (NFR6, NFR7).

1.5 Sécurité & protection des données

Sécurité **par conception**, en défense en profondeur. Le détail par couche figure en parties IoT (collier), Cloud (infrastructure) et IA (Care Engine) ; voici la synthèse des frontières :

- **Frontières de confiance** : depuis Internet, tout passe par la passerelle unique (KrakenD : validation du JWT en bordure, limitation de débit) ; l'IoT entre par un broker MQTT séparé (TLS + certificat par collier, PKI step-ca) ; en interne, les services communiquent en **mTLS**.
- **Identité & moindre privilège** : OIDC avec des scopes distincts propriétaire / vétérinaire / admin. Un vétérinaire n'accède qu'aux animaux qui lui sont rattachés.
- **Chiffrement** : en transit (TLS/mTLS partout) et au repos (volumes chiffrés) ; secrets versionnés chiffrés (SOPS + age), jamais en clair dans le dépôt.
- **Données de santé & RGPD** : minimisation (on ne collecte que l'utile), rétention par finalité, résidence UE (hébergement et LLM Azure en région UE). Le droit à l'effacement s'applique aux données métier ; le journal d'audit fait exception, conservé pour la preuve juridique (voir ci-dessous).
- **IA cloisonnée** : le Care Engine n'a aucun accès en écriture au métier (ADR-001).

1.6 Traçabilité & journal d'audit

Le risque juridique majeur du produit : qu'un client affirme que l'IA a **posé un diagnostic** et nous en tienne pour responsables (par exemple le décès de l'animal). La parade est architecturale : le Care Engine **journalise l'intégralité de chaque conversation** (entrée, passages récupérés par le RAG, sortie, décisions des garde-fous) dans un journal **append-only conservé 5 ans**. En cas de litige, on prouve mot pour mot ce qui a été dit : une orientation ou une escalade, **jamais** un diagnostic (Code rural, art. L243-1 ; ADR-010).

- **Contenu = conversations complètes** : pas de simples métadonnées ; chaque échange IA est rejouable tel qu'il s'est produit. Ces enregistrements servent aussi de base d'évaluation qualité du Care Engine.
- **Immuable (WORM)** : stockage objet append-only avec **Object Lock** (S3-compatible) : écriture seule, ni modification ni suppression pendant la durée de rétention.
- **Rétention 5 ans** : alignée sur le délai utile à la preuve en cas de mise en cause.

- **Périmètre étendu** : au-delà des conversations, les accès vétérinaire aux données d'un animal et les escalades sont également tracés.
- **Articulation RGPD** : ces échanges contiennent des données personnelles ; leur conservation malgré une demande d'effacement est justifiée par la **constitution de preuve en vue d'une action en justice** (RGPD, art. 17-3-e). Accès strictement restreint et lui-même journalisé.